

Extra 1 · CTEs and Recursive Queries

Why this matters in interviews. "Rewrite this with a CTE", "print the org chart", "generate the numbers 1 to 10 / every date in a month with no table", "find all subordinates of a manager" — these are standard mid-level questions, and the `emp` table (each employee has a `manager_id`) is exactly the shape they are asked on.

In this chapter: `WITH` (CTE) basics · chaining several CTEs · CTE vs subquery vs view vs temp table · recursive CTEs (anchor + recursive member) · Oracle's older `CONNECT BY` · classic patterns · cycles · dialect differences.

E1.1 What a CTE is

A **Common Table Expression** gives a **name to a subquery** that lasts for **one statement**. In Oracle the official name is **subquery factoring**.

```
WITH name [(col1, col2, ...)] AS (  
    SELECT ...  
)  
SELECT ... FROM name ...;
```

Think of it as a **temporary, named result set** you define at the top and then use like a table underneath. It is not stored anywhere and disappears when the statement finishes.

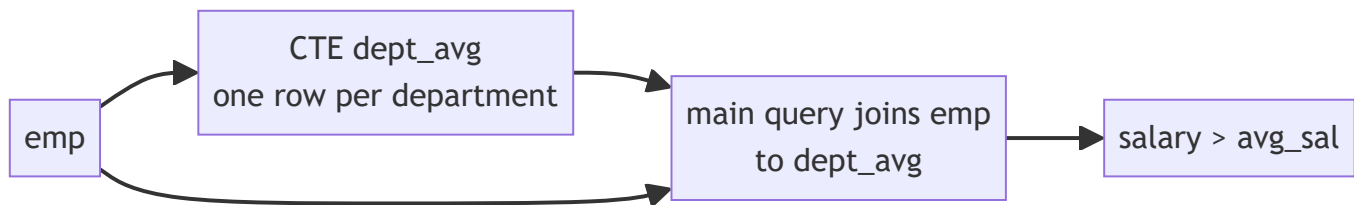
Example — employees who earn more than their department's average

Without a CTE the average is buried in a nested subquery. With one, the logic reads top to bottom:

```
WITH dept_avg AS (  
    SELECT dept_id, AVG(salary) AS avg_sal  
    FROM emp  
    WHERE dept_id IS NOT NULL  
    GROUP BY dept_id  
)  
SELECT e.emp_name, e.salary, d.avg_sal  
FROM emp e  
JOIN dept_avg d ON d.dept_id = e.dept_id  
WHERE e.salary > d.avg_sal  
ORDER BY e.emp_name;
```

Result

emp_name	salary	avg_sal
Bala	70000	46000
Chitra	50000	46000
Dinesh	50000	46000
Farhan	65000	45000



Where a CTE can be used. Only in the statement it belongs to; it must come **first** (`WITH` before `SELECT`). Oracle allows `WITH` before a `SELECT` (including inside `INSERT ... SELECT` and subqueries); a `WITH` directly in front of `UPDATE` or `DELETE` is **not** allowed in Oracle — put the CTE inside a subquery instead. (PostgreSQL and SQL Server do allow it.)

E1.2 Several CTEs, each using the previous ones

Separate them with commas — **one** `WITH` , **many names**. A later CTE can refer to any earlier one.

```

WITH dept_totals AS (
    SELECT dept_id, SUM(salary) AS total
    FROM emp
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
),
company AS (
    SELECT SUM(total) AS grand_total FROM dept_totals
)
SELECT d.dept_name,
       t.total,
       ROUND(100 * t.total / c.grand_total, 1) AS pct_of_company
FROM dept_totals t
JOIN dept d ON d.dept_id = t.dept_id
CROSS JOIN company c
ORDER BY t.total DESC;
  
```

Result

dept_name	total	pct_of_company
Engineering	230000	50.5
Sales	135000	29.7
Management	90000	19.8

Each block does one small job (per-department totals → company total → percentage). That is the real value of a CTE: **a long query becomes a sequence of readable steps**.

Using the same CTE twice

A subquery would have to be repeated; a CTE is written once and referenced as many times as needed.

```

WITH dept_avg AS (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM emp
    WHERE dept_id IS NOT NULL
    GROUP BY dept_id
)
SELECT 'highest' AS which, dept_id, avg_sal FROM dept_avg
WHERE avg_sal = (SELECT MAX(avg_sal) FROM dept_avg)
UNION ALL
SELECT 'lowest', dept_id, avg_sal FROM dept_avg
WHERE avg_sal = (SELECT MIN(avg_sal) FROM dept_avg);

```

Result

which	dept_id	avg_sal
highest	10	90000
lowest	30	45000

Oracle note. When Oracle sees a CTE referenced more than once it may **materialise** it (compute once into a temporary area). You can force it with the hint `/*+ MATERIALIZE */` or forbid it with `/*+ INLINE */`. Both hints are optional tuning tools — the *result* is identical either way.

E1.3 CTE vs subquery vs view vs temporary table

	Inline subquery	CTE	View	Temporary table
Lives for	the statement	the statement	permanently (definition only)	a session or transaction
Stores data?	no	no (may be materialised internally)	no	yes
Named & reusable in the statement	✗	✓	✓	✓
Reusable by <i>other</i> statements	✗	✗	✓	✓
Can be recursive	✗	✓	✗	✗
Best for	one-off filter	readability, steps, recursion	shared logic, security	large intermediate results reused many times

A CTE is not automatically faster. It is mostly a *readability* tool. Modern optimisers usually treat it exactly like the equivalent subquery. Don't claim "CTEs are faster" in an interview.

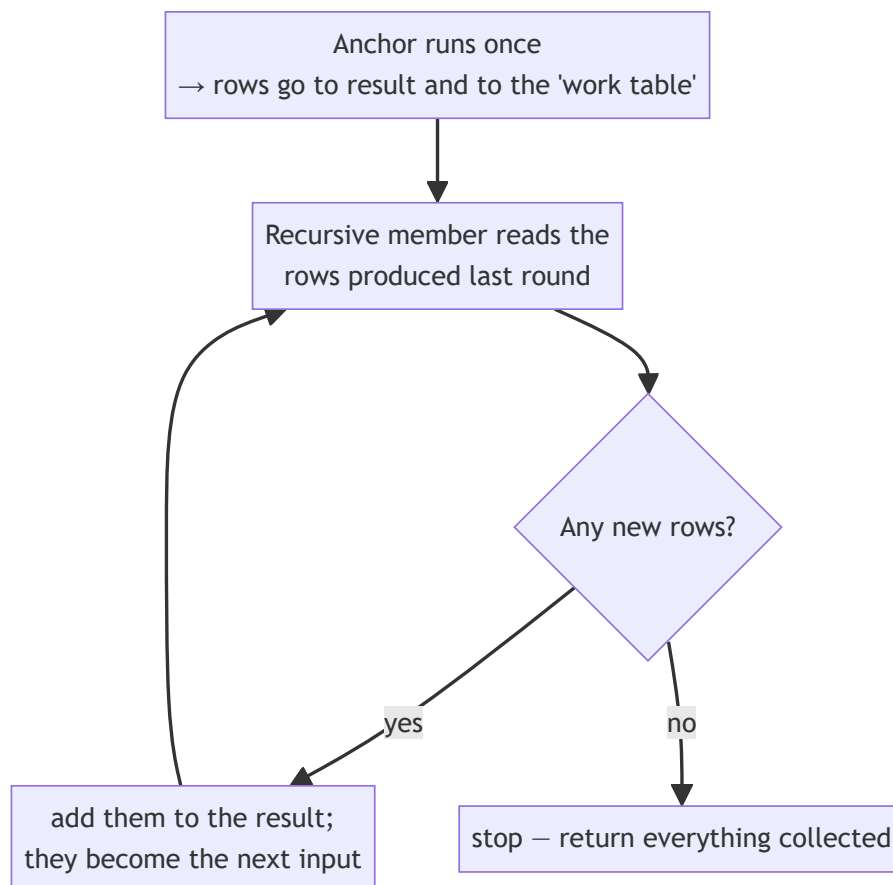
E1.4 Recursive CTEs

A **recursive CTE** is one that **refers to itself**. It is the standard way to walk a hierarchy (org charts, category trees, bill of materials) or to generate a series.

Anatomy

```
WITH name (col1, col2, ...) AS (  
    -- 1. ANCHOR member: the starting rows (runs once)  
    SELECT ...  
    UNION ALL  
    -- 2. RECURSIVE member: refers to 'name'; runs repeatedly  
    SELECT ... FROM name JOIN ...  
)  
SELECT ... FROM name;
```

How it executes:



Two hard rules: the two members are joined with **UNION ALL**, and the recursive member must eventually return **no rows**, otherwise it loops forever.

Oracle vs others. Oracle (11gR2 and later) writes just **WITH** and requires a **column list** after the name. PostgreSQL and MySQL 8 write **WITH RECURSIVE**. SQL Server writes just **WITH** (no **RECURSIVE**, and the column list is optional).

Example 1 — generate the numbers 1 to 10

```
WITH numbers (n) AS (  
    SELECT 1 FROM dual          -- anchor  
    UNION ALL  
    SELECT n + 1 FROM numbers   -- recursive member  
    WHERE  n < 10              -- stop condition  
)  
SELECT n FROM numbers;
```

Result

n
1
2
3
4
5
6
7
8
9
10

Round 0 gives 1; round 1 reads 1 and produces 2; ... round 9 reads 9 and produces 10; round 10 reads 10, the WHERE fails, no rows, stop. **Without the WHERE n < 10 this never ends.**

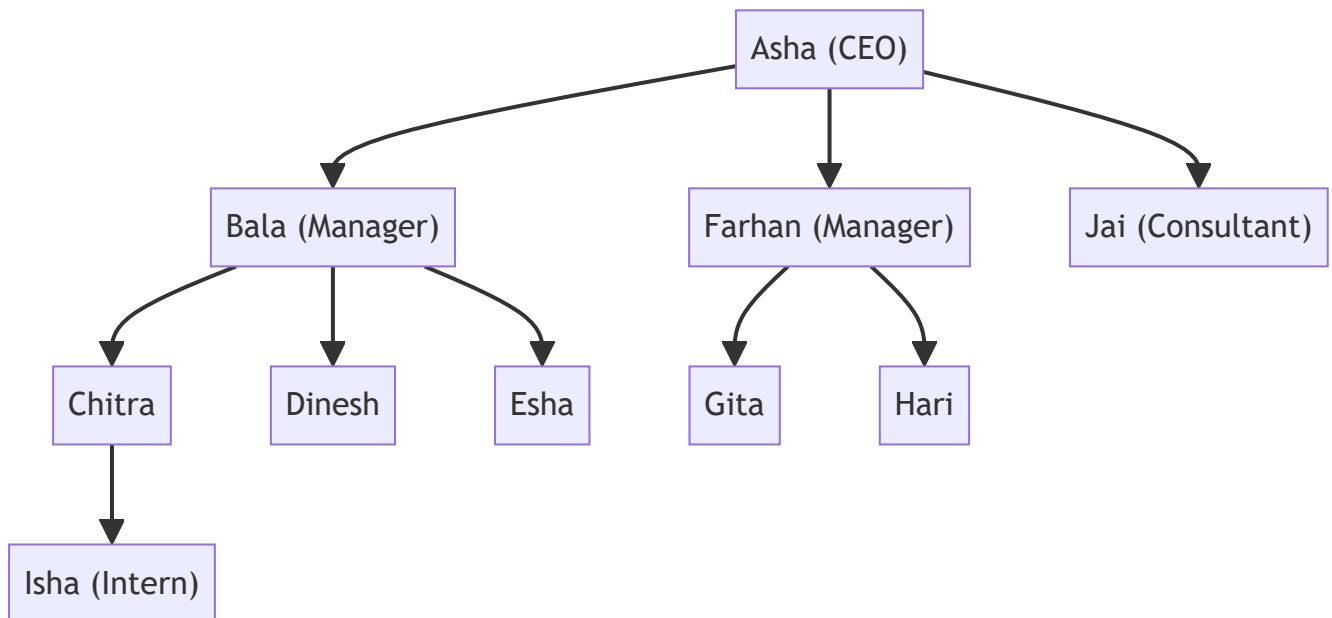
Example 2 — the org chart, with level and path

The anchor is the person with no manager; each round finds the people who report to the previous round.

```
WITH org (emp_id, emp_name, manager_id, lvl, path) AS (  
    SELECT emp_id, emp_name, manager_id, 1, emp_name  
    FROM emp  
    WHERE manager_id IS NULL  
    UNION ALL  
    SELECT e.emp_id, e.emp_name, e.manager_id, o.lvl + 1, o.path || ' > ' || e.emp_name  
    FROM emp e  
    JOIN org o ON e.manager_id = o.emp_id  
)  
SELECT emp_id, emp_name, lvl, path  
FROM org  
ORDER BY path;
```

Result

emp_id	emp_name	lvl	path
101	Asha	1	Asha
102	Bala	2	Asha > Bala
103	Chitra	3	Asha > Bala > Chitra
109	Isha	4	Asha > Bala > Chitra > Isha
104	Dinesh	3	Asha > Bala > Dinesh
105	Esha	3	Asha > Bala > Esha
106	Farhan	2	Asha > Farhan
107	Gita	3	Asha > Farhan > Gita
108	Hari	3	Asha > Farhan > Hari
110	Jai	2	Asha > Jai



`lvl` is the depth (CEO = 1); `path` is built up one name at a time — a very common interview follow-up ("show the reporting chain").

Example 3 — indented tree

```
WITH org (emp_id, emp_name, lvl, sort_key) AS (  
    SELECT emp_id, emp_name, 1, emp_name  
    FROM emp  
    WHERE manager_id IS NULL  
    UNION ALL  
    SELECT e.emp_id, e.emp_name, o.lvl + 1, o.sort_key || '/' || e.emp_name  
    FROM emp e  
    JOIN org o ON e.manager_id = o.emp_id  
)  
SELECT LPAD(' ', 4 * (lvl - 1)) || emp_name AS org_chart  
FROM org  
ORDER BY sort_key;
```

Result

org_chart
Asha
Bala
Chitra
Isha
Dinesh
Esha
Farhan
Gita
Hari
Jai

`LPAD(' ', 4*(lvl-1))` pads with spaces proportional to the depth, and sorting on the accumulated `sort_key` keeps every child directly under its parent.

E1.5 Oracle's original way — `CONNECT BY`

Before recursive CTEs, Oracle had its own hierarchical query syntax. It is **Oracle-only** but still extremely common in Oracle shops, and interviewers for Oracle roles will ask it.

```
SELECT LEVEL, emp_name, manager_id  
FROM emp  
START WITH manager_id IS NULL          -- where the tree begins (the root rows)  
CONNECT BY PRIOR emp_id = manager_id  -- how a parent row links to its child rows  
ORDER SIBLINGS BY emp_name;          -- sort only among rows of the same parent
```

Piece	Meaning
<code>START WITH cond</code>	the root rows (the anchor). Omit it and <i>every</i> row becomes a root
<code>CONNECT BY ... PRIOR ...</code>	the link. PRIOR marks the parent side : <code>PRIOR emp_id = manager_id</code> means "child's <code>manager_id</code> = parent's <code>emp_id</code> " (walk down). Swap it — <code>emp_id = PRIOR manager_id</code> — to walk up
<code>LEVEL</code>	depth of the row (root = 1)
<code>ORDER SIBLINGS BY</code>	orders children within each parent while keeping the tree shape
<code>SYS_CONNECT_BY_PATH(col, '/')</code>	the path from the root, e.g. <code>/Asha/Bala/Chitra</code>
<code>CONNECT_BY_ISLEAF</code>	1 for a row with no children
<code>CONNECT_BY_ROOT col</code>	the value of <code>col</code> on the root row of this branch
<code>NOCYCLE + CONNECT_BY_ISCYCLE</code>	survive (and detect) loops in the data

Full example with the useful pseudo-columns (the result shown is the same tree as Example 2):

```
SELECT LEVEL AS lvl,
       LPAD(' ', 2 * (LEVEL - 1)) || emp_name AS emp_name,
       SYS_CONNECT_BY_PATH(emp_name, '/')      AS path,
       CONNECT_BY_ISLEAF                      AS is_leaf
FROM   emp
START WITH manager_id IS NULL
CONNECT BY PRIOR emp_id = manager_id
ORDER  SIBLINGS BY emp_name;
```

Result

lvl	emp_name	path	is_leaf
1	Asha	/Asha	0
2	Bala	/Asha/Bala	0
3	Chitra	/Asha/Bala/Chitra	0
4	Isha	/Asha/Bala/Chitra/Isha	1
3	Dinesh	/Asha/Bala/Dinesh	1
3	Esha	/Asha/Bala/Esha	1
2	Farhan	/Asha/Farhan	0
3	Gita	/Asha/Farhan/Gita	1
3	Hari	/Asha/Farhan/Hari	1
2	Jai	/Asha/Jai	1

Verification note. Oracle is not available where these notes were built, so this `CONNECT BY` result was produced from the equivalent recursive CTE above. Siblings are ordered alphabetically, which is what `ORDER SIBLINGS BY emp_name` does.

The one-line number generator (Oracle only, very common)

```
SELECT LEVEL AS n FROM dual CONNECT BY LEVEL <= 10;    -- 1, 2, ... 10
```

With no `PRIOR` and no `START WITH`, `CONNECT BY LEVEL <= 10` simply loops ten times from a one-row table. It is the Oracle idiom for "generate rows out of nothing" — dates in a range, numbering, test data.

```
-- every day in August 2025
SELECT DATE '2025-08-01' + LEVEL - 1 AS day
FROM   dual
CONNECT BY LEVEL <= 31;
```

WHERE before or after CONNECT BY ? A common trap

- A `WHERE` in a hierarchical query is applied **after** the tree is built, to the resulting rows.
- To prune **branches** while walking, put the condition in `CONNECT BY` (e.g. `CONNECT BY PRIOR emp_id = manager_id AND salary > 30000`).

E1.6 Classic patterns

All the subordinates of a given manager (any depth)

"Everyone under Bala."

```
WITH under AS (
  SELECT emp_id, emp_name, manager_id, 1 AS depth
  FROM   emp
  WHERE  manager_id = 102          -- Bala's direct reports
  UNION ALL
  SELECT e.emp_id, e.emp_name, e.manager_id, u.depth + 1
  FROM   emp e
  JOIN   under u ON e.manager_id = u.emp_id
)
SELECT emp_name, depth FROM under ORDER BY depth, emp_name;
```

Result

emp_name	depth
Chitra	1
Dinesh	1
Esha	1
Isha	2

Isha is shown at depth 2 — she reports to Chitra, who reports to Bala. A plain self-join finds only one level.

How many people report (directly or not) to each manager?

```
WITH chain (top_id, emp_id) AS (  
    SELECT emp_id, emp_id FROM emp  
    UNION ALL  
    SELECT c.top_id, e.emp_id  
    FROM   chain c  
    JOIN   emp e ON e.manager_id = c.emp_id  
)  
SELECT m.emp_name, COUNT(*) - 1 AS total_reports  
FROM   chain c  
JOIN   emp m ON m.emp_id = c.top_id  
GROUP BY m.emp_name  
HAVING COUNT(*) - 1 > 0  
ORDER BY total_reports DESC, m.emp_name;
```

Result

emp_name	total_reports
Asha	9
Bala	4
Farhan	2
Chitra	1

The anchor makes every employee the "top" of their own chain; each round extends the chain one level down. Counting rows per top and subtracting the person themselves gives the total headcount below them.

Walking up — the chain of command from an employee to the CEO

```
WITH up (emp_id, emp_name, manager_id, step) AS (  
    SELECT emp_id, emp_name, manager_id, 0  
    FROM   emp  
    WHERE  emp_name = 'Isha'  
    UNION ALL  
    SELECT e.emp_id, e.emp_name, e.manager_id, u.step + 1  
    FROM   emp e  
    JOIN   up u ON e.emp_id = u.manager_id  
)  
SELECT step, emp_name FROM up ORDER BY step;
```

Result

step	emp_name
0	Isha
1	Chitra
2	Bala

step	emp_name
3	Asha

Same pattern, join reversed: the next row is the *manager* of the current one.

The depth of the hierarchy

```
WITH org (emp_id, lvl) AS (
  SELECT emp_id, 1 FROM emp WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, o.lvl + 1 FROM emp e JOIN org o ON e.manager_id = o.emp_id
)
SELECT MAX(lvl) AS levels FROM org;
```

Result

levels
4

Filling gaps — every month even when there were no sales

Recursive CTEs are the standard trick for a **calendar you don't have**: generate the months, then `LEFT JOIN` the real data so empty months show as 0.

```
WITH months (m) AS (
  SELECT DATE '2025-01-01' FROM dual
  UNION ALL
  SELECT ADD_MONTHS(m, 1) FROM months
  WHERE m < DATE '2025-08-01'
)
SELECT TO_CHAR(m, 'YYYY-MM') AS month,
       NVL(s.amount, 0) AS amount
FROM   months
LEFT JOIN monthly_sales s ON s.month = TO_CHAR(months.m, 'YYYY-MM')
ORDER BY m;
```

Result

month	amount
2025-01	100
2025-02	120
2025-03	90
2025-04	150
2025-05	150
2025-06	210
2025-07	0

month	amount
2025-08	0

The table only has January to June; the generated calendar adds July and August as zero rows. Without the calendar, the missing months would simply not appear.

E1.7 Cycles — the way recursion goes wrong

If the data loops (A reports to C, C to B, B to A), the recursive member never runs out of rows.

staff2

emp_id	emp_name	manager_id
1	A	3
2	B	1
3	C	2

Here A → manager C → manager B → manager A. Walking up from A would go round forever. Every database has a defence:

Database	How to stop it
Oracle	<code>CYCLE col SET is_cycle TO 'Y' DEFAULT 'N'</code> clause on a recursive <code>WITH</code> ; or <code>NOCYCLE</code> in <code>CONNECT BY</code> . Otherwise the error ORA-32044: cycle detected while executing recursive WITH query (and ORA-01436 for <code>CONNECT BY</code>)
PostgreSQL 14+	<code>CYCLE</code> clause; older versions: keep a visited-array and test it
SQL Server	default limit 100 recursions → error 530; change with <code>OPTION (MAXRECURSION n)</code>
MySQL 8	<code>cte_max_recursion_depth</code> (default 1000)

A portable defence is a **depth guard** in the recursive member — `WHERE lvl < 20` — plus, ideally, cleaning the data.

E1.8 CTE dialect differences at a glance

	Oracle	MySQL 8	PostgreSQL	SQL Server
Basic <code>WITH</code>	✅ (9i+)	✅ (8.0+, not 5.7)	✅	✅ (2005+)
Recursive keyword	<code>WITH</code> (11gR2+)	<code>WITH RECURSIVE</code>	<code>WITH RECURSIVE</code>	<code>WITH</code>
Column list required	✅ yes	optional	optional	optional
Own hierarchy syntax	<code>CONNECT BY</code>	❌	❌ (<code>ltree</code> extension)	<code>hierarchyid</code> type
Number generator	<code>CONNECT BY LEVEL <= n</code>	recursive CTE	<code>generate_series(1, n)</code>	recursive CTE / <code>GENERATE_SERIES</code> (2022)

	Oracle	MySQL 8	PostgreSQL	SQL Server
<code>WITH</code> before <code>UPDATE / DELETE</code>	✗	✓	✓ (even data-modifying CTEs)	✓

Interview corner

Q. What is a CTE and why use one? A named temporary result set defined with `WITH` for a single statement. It makes complex queries readable by splitting them into steps, lets one result be referenced several times, and is the only way (besides `CONNECT BY` in Oracle) to write recursive queries.

Q. CTE vs subquery vs view vs temp table? Subquery: inline, anonymous. CTE: named, one statement, may be recursive. View: named and stored permanently, reusable across statements. Temp table: actually stores data, reusable across statements in the session.

Q. Is a CTE faster than a subquery? No, not by itself — it is mostly about readability; the optimiser usually treats them the same. It can avoid repeated work if it is materialised.

Q. How does a recursive CTE work? An anchor member seeds the result; a recursive member repeatedly joins the previous round's rows to the source until it returns no rows; all rounds are combined with `UNION ALL`.

Q. What is the danger of recursion and how do you prevent it? Infinite loops from a missing stop condition or cyclic data. Add a depth limit, use the database's cycle-detection feature (`CYCLE / NOCYCLE` , `MAXRECURSION`), and fix the data.

Q. Explain `CONNECT BY PRIOR emp_id = manager_id`. Each child row's `manager_id` must equal its parent's `emp_id`. `PRIOR` refers to the parent row. Swapping the sides walks up the tree instead of down.

Q. Print the org chart with indentation. `LPAD(' ', 4*(LEVEL-1)) || emp_name` with `START WITH manager_id IS NULL CONNECT BY PRIOR emp_id = manager_id ORDER SIBLINGS BY emp_name`.

Q. Generate 1 to 10 without a table. Oracle: `SELECT LEVEL FROM dual CONNECT BY LEVEL <= 10`. Others: a recursive CTE.

Quick recap

- `WITH name AS (...)` = named, single-statement result set; chain several with commas; readability tool, not a speed tool.
- Recursive CTE = **anchor** `UNION ALL` **recursive member** + a stop condition; Oracle omits `RECURSIVE` and needs the column list.
- Oracle `CONNECT BY`: `START WITH root, PRIOR` = parent side, `LEVEL`, `SYS_CONNECT_BY_PATH`, `ORDER SIBLINGS BY`, `CONNECT BY LEVEL <= n` generator.
- Walk **down** for subordinates, **up** for the chain of command; guard against **cycles**.
- `WITH` cannot directly precede `UPDATE / DELETE` in Oracle.